Reto:

Transformer Variacional para el modelo de Ising

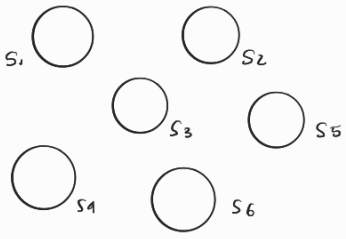
Valentina Pérez Cadavid

2026

A. Principio Variacional de Gibss

Se tiene un sistema de espines cuya distribución de equilibrio es la de Boltzmann

$$p_B(s) = \frac{e^{-\beta E_0(s)}}{Z}$$



Dado que Z es una suma sobre muchos términos $\sim 2^N$, entonces se entrena una red (Transformer) que define otra distribución $q_\theta(s)$ que se debe ir ajustando para que se parezca a p_B y que permita generar nuevas configuraciones representativas del sistema

Para esto, minimizar la energía libre $F[q_\theta]$ equivale a acercar q_θ a p_B sin necesidad de conocer Z.

El modelo parametrizable $q_\theta(s)$, puede representarse mediante el producto de probabilidades condicionales

$$q_\theta(s) = \prod_{i=1}^{N} q_\theta(s_i \mid s_1, \dots s_{i-1})$$

con θ: parámetros de la red neuronal

Se deben ajustar los parámetros θ para que $q_\theta(s)$ se aproxime a $p_B(s)$. Para cuantificar esto se usa la divergencia de Kullback-Leiber:

$$D_{KL}(q_\theta \| p_B) = \sum_s q_\theta(s) \ln \frac{q_\theta(s)}{p_B(s)}$$

$$= \sum_s q_\theta(s) \ln q_\theta(s) - \sum_s q_\theta(s) \ln p_B(s)$$

$$= \sum_s q_\theta(s) \ln q_\theta(s) - \sum_s q_\theta(s) \ln \left(\frac{e^{-\beta E_0(s)}}{Z} \right)$$

$$= \sum_s q_\theta(s) \ln q_\theta(s) - \sum_s q_\theta(s) \left[\ln e^{-\beta E_\theta(s)} - \ln Z \right]$$

$$= \sum_s q_\theta(s) \ln q_\theta(s) + \beta \sum_s q_\theta(s) E_\theta(s) + \ln Z \sum_s q_\theta(s)$$

es una distrib de probabilidad

$$= \sum_s q_\theta(s) \ln q_\theta(s) + \beta \sum_s q_\theta(s) E_\theta(s) + \ln Z$$

ẽ promedio evaluada sobre la distribución q

$$= \sum_s q_\theta(s) \ln q_\theta(s) + \beta \langle E \rangle_q + \ln Z$$

$$= \frac{-K_B}{-K_B} \sum_s q_\theta(s) \ln q_\theta(s) + \beta \langle E \rangle_q + \ln Z$$

ẽ de Gibbs-Shannon $S[q]$

$$= - \frac{S[q]}{K_B} + \beta \langle E \rangle_q + \ln Z$$

$$= - \beta T S[q] + \beta \langle E \rangle_q + \ln Z$$

$$= \beta \left(\langle E \rangle_q - T S[q] \right) + \ln Z \left(\frac{-\beta}{-\beta} \right)$$

ẽ libre de equilibrio F_θ

ẽ libre de Helmholtz $F[q]$

$$= \beta F[q] - \beta F_\theta$$ ⇝ Divergencia

Como $D_{KL}(q \| p_\theta) \geq 0$ entonces: $\beta F[q] - \beta F_\theta \geq 0 \rightarrow \boxed{F[q] \geq F_\theta}$

Cualquier distribución q que se construya tiene una ẽ mayor a la verdadera

$D_{KL}(q \| p_\theta) = 0$ si las 2 distribuciones son idénticas $q_\theta = p_\theta$, entonces: $F_\beta = \ln Z(-\beta)$

$$F[q] = \left(\langle E \rangle_q - T S[q] \right)$$

$$= \sum_s q(s) E(s) - T \left(-k \sum_s q(s) \ln q(s) \right)$$

$$= \sum_s q(s) E(s) + T k \sum_s q(s) \ln q(s)$$

Cuando $q = p_\beta$:

$$F[p_\theta] = \sum_s p_\beta(s) E(s) + T k \sum_s p_\beta(s) \ln p_\beta(s)$$

$$= \langle E \rangle + T k \sum_s p_\beta(s) \ln p_\beta(s)$$

$$p_B(s) = \frac{e^{-\beta E(s)}}{Z} \rightarrow \ln p_B(s) = \ln e^{-\beta E(s)} - \ln Z = -\beta E(s) - \ln Z$$

$$= \langle E \rangle + TK \sum_s p_B(s) (-\beta E(s) - \ln Z)$$

$$= \langle E \rangle + TK \sum_s (-\beta p_B(s) E(s) - p_B(s) \ln Z)$$

$$= \langle E \rangle + TK \left(-\beta \sum_s p_B(s) E(s) - \ln Z \sum_s p_B(s) \right)$$

$$= \langle E \rangle + TK (-\beta \langle E \rangle - \ln Z)$$

$$= \langle E \rangle - TK\beta \langle E \rangle - TK \ln Z \qquad \text{con} \quad T\beta = T \cdot \frac{1}{T} = 1 \text{, omitiendo } K$$

$$= \langle E \rangle - \langle E \rangle - T \ln Z$$

$$= -T \ln Z = -T(-\beta F_B) = F_B$$

Por tanto: $\boxed{F[p_B] = F_B}$ Por tanto $F_B = \min F[q]$

La distribución de Boltzmann es la que minimiza la energía libre.

Minimizar $F[q_\theta]$ equivale a acercar el modelo a p_B sin conocer Z.

$$\theta^* = \arg\min F[q_\theta] = \arg\min D_{KL}(q_\theta \,||\, p_B)$$

Los mejores parámetros del Transformer son los que hacen mínima su energía libre F y estos mismos parámetros son los que hacen que su distribución se parezca lo más posible a la de Boltzmann.

El modelo q_θ genera configuraciones del sistema, y sus parámetros se ajustan buscando un equilibrio entre reducir la energía media y aumentar la entropía, según la T.

$$F[q] = \sum_s q(s) E(s) + TK \sum_s q(s) \ln q(s)$$

Para minimizar la energía libre respecto a los parámetros del modelo y asumiendo que T y $E(s)$ no dependen de θ:

$$\nabla_\theta F[q_\theta] = \nabla_\theta \left[\sum_s q(s) E(s) + TK \sum_s q(s) \ln q(s) \right]$$

$$= \sum_s E(s) \nabla_\theta q(s) + TK \sum_s \nabla_\theta \left(q_\theta(s) \ln q_\theta(s) \right)$$

$$= \sum_s E(s) \nabla_\theta q(s) + TK \sum_s \left(\nabla_{q_\theta(s)} \cdot \ln q_\theta(s) + q_\theta(s) \nabla_{q_\theta} \ln q_\theta(s) \right)$$

$$= \sum_s E(s) \nabla_\theta q(s) + TK \sum_s \left(\nabla q_\theta(s) \ln q_\theta(s) + \cancel{q_\theta(s)} \cdot \frac{1}{\cancel{q_\theta(s)}} \nabla_\theta q_\theta(s) \right)$$

$$= \sum_s \nabla_\theta q_\theta(s) \left[E(s) + TK \left(\ln q_\theta(s) + 1 \right) \right]$$

$$= \sum_s \nabla_\theta q_\theta(s) \left[E(s) + TK \ln q_\theta(s) \right] \rightsquigarrow \text{Haciendo} \quad \nabla q_\theta(s) = \frac{q_\theta(s)}{q_\theta(s)} \nabla q_\theta(s)$$

$$= q_\theta(s) \nabla \ln q_\theta(s)$$

$$= \sum_s q_\theta(s) \nabla_\theta \ln q_\theta(s) \left[E(s) + TK \ln q_\theta(s) \right]$$

$$= \sum_s q_\theta(s) \cdot \left[E(s) + TK \ln q_\theta(s) \right] \cdot \nabla_\theta \ln q_\theta(s)$$

De manera general:

$$\langle g \rangle_q = \sum_s q(s) g(s)$$

$$\boxed{\nabla_\theta F[q_\theta] = \left\langle \left[E(s) + TK \ln q_\theta(s) \right] \cdot \nabla_\theta \ln q_\theta(s) \right\rangle_{q_\theta}}$$

Un mínimo interior satisface: $\nabla_\theta F[q_\theta] = 0$ ⇝ Dado que F depende de todos los parámetros no hay una forma de despejarlos. Por tanto, se usa la opción iterativa:

$$\boxed{\theta_{t+1} = \theta_t - \eta \nabla_\theta F[q_\theta]}$$

$\eta > 0$ Tasa de aprendizaje

B. El Transformer como distribución variacional

Se usará un Transformer autoregresivo para representar la distribución $q_\theta(s)$ de un sistema de N espines $s_i \in \{-1, 1\}$. De este modo, el Transformer genera la configuración espín por espín en orden, recibe los espines anteriores y devuelve la probabilidad del siguiente. La probabilidad total es el producto de cada probabilidad de un espín dado los espines anteriores.

$$q_\theta(s) = \prod_{i=1}^{N} q_\theta(s_i \mid s_{<i}) = q_\theta(s_1) \cdot q_\theta(s_2 \mid s_1) \cdot q_\theta(s_3 \mid s_1, s_2) \cdots$$

1. Cálculo de las probabilidades: Los valores +1 y -1 se representan mediante tokens y se añade un token de inicio BOS. Por tanto, se usa la secuencia de entrada desplazada:

$$[1, 1, 1, 1, -1, 1, 1, 1, 1]$$
$$\downarrow$$
$$[1, 1, 1, 0, 1, 1, 1, 1]$$
$$\downarrow$$
$$[\text{BOS}, 1, 1, 1, 0, 1, 1, 1, 1]$$

Los embeddings de los tokens se combinan con información de posición. La máscara causal permite que la salida en la posición i dependa únicamente del token de inicio y de los espines $s_{<i}$ (ver partes 1, 2, 3 del Transformer de Juguete)

Si $h_i \in \mathbb{R}^d$ es la representación interna en esa posición, la capa de salida calcula los logits:

$$\boxed{z_i = W_{out} h_i + b} \quad ; \quad W_{out} \in \mathbb{R}^{2 \times d}$$

(ver parte 5 del Transformer de Juguete)

Luego, la función softmax los transforma en probabilidades:

$$q_\theta(s_i = a \mid s_{<i}) = \frac{e^{z_i^{(a)}}}{e^{z_i^{(-1)}} + e^{z_i^{(+1)}}} \qquad a \in \{-1, 1\}$$

(ver parte 4 del Transformer de Juguete)

Repitiendo este procedimiento se obtiene una configuración $s \sim q_\theta$.
Para calcular su logaritmo de probabilidad, se selecciona en cada posición la probabilidad del espín observado y se suma:

$$\ln q_\theta(s) = \sum_{i=1}^{N} \ln q_\theta(s_i \mid s_{<i})$$

(ver parte 7 del Transformer de Juguete)

2. Estimación de la energía Libre: En cada iteración se genera un lote de M configuraciones independientes con los parámetros actuales: $s^{(1)}, \dots, s^{(M)} \sim q_\theta$

Para cada configuración calculamos 2 cantidades

$$E_m = E(s^{(m)}) \qquad l_m = \ln q_\theta(s^{(m)})$$

Considerando una red de espines $L \times L$ con condiciones periódicas, $J = 1$ y campo externo $h = 0$:

$$E_m = -\sum_{\langle ij \rangle} s_i^{(m)} s_j^{(m)}$$

De este modo, se calcula la energía libre de cada configuración:

$$f_m = E_m + T l_m$$

El promedio del lote proporciona un estimador de la energía libre:

$$\hat{F}_\theta = \frac{1}{M} \sum_{m=1}^{M} f_m$$

3. Actualización de los parámetros: El gradiente de la energía libre estimado con el lote, la dirección en la que hay que mover los parámetros, calculada con las M configuraciones que generó el Transformer

$$\hat{g}_\theta = \frac{1}{M} \sum_{m=1}^{M} f_m \nabla_\theta l_m$$

Para implementarla, se debe construir la función auxiliar:

$$\mathcal{L}_{max}(\theta) = \frac{1}{M} \sum_{m=1}^{M} \text{stopgrad}(f_m) \, l_m$$

Así: $\nabla_\theta \mathcal{L}_{max} = \hat{g}_\theta$

Los parámetros se actualizan mediante:

$$\theta_{t+1} = \theta_t - \eta \hat{g}_\theta$$

o mediante un optimizador como Adam

El ciclo de entrenamiento consiste en generar configuraciones, evaluar sus energías y logaritmos de probabilidad, calcular el gradiente y actualizar el Transformer. En la sig. iteración, el nuevo lote se genera con los parámetros actualizados

Los parámetros θ del Transformer, incluye:

- Embeddings: convierte un espín y su posición en un vector de #s
- Bloques de Atención: cada posición mira los espines anteriores y decide cuáles importan más. De este modo, la posición i queda representada por un vector h_i que resume $s_{<i}$. eg $h_i = [0.5, -1, 2]$

- Capa de salida: convierte el vector interno en 2 #s (logits = un puntaje) y con softmax en las probabilidades de que el siguiente espín sea -1 y la de que sea 1.

$$z_i = W h_i + b \quad ; \quad W \in \mathbb{R}^{2 \times d}$$

Transformer de Juguete

1. Suponer una configuración dada por: $S = \begin{pmatrix} 1 & 1 & 1 \\ 1 & -1 & 1 \\ 1 & 1 & 1 \end{pmatrix} = [1, 1, 1, 1, -1, 1, 1, 1, 1]$

2. Convertir a Tokens (porque el Transformer solo recibe # números enteros (tokens)

valor	Token
-1	0
1	1
BOS	2

$\longrightarrow [1, 1, 1, 0, 1, 1, 1, 1] \rightarrow N = 9$ tokens

N: # espines tokens

d: Tamaño del vector con que se representa cada token (se escoge a mano)

3. Embedding de Token: cambia cada token por un vector de $d = 2$ #

Son p.θ.

Posición	Entrada	Token	Vector ($d = 2$)
1	BOS	2	[0, 1]
2	$s_1 = +1$	1	[1, 0]
3	$s_2 = +1$	1	[1, 0]
4	$s_3 = +1$	1	[1, 0]
5	$s_4 = +1$	1	[1, 0]
6	$s_5 = -1$	0	[−1, 0]
7	$s_6 = +1$	1	[1, 0]
8	$s_7 = +1$	1	[1, 0]
9	$s_8 = +1$	1	[1, 0]

Posición (son parámetros θ)

[0, 0]
[0, 0.2]
[,]
[,]
[,]
[,]
[,]
[,]
[,]

$[2, 1, 1, 1, 1, 0, 1, 1, 1] = [\underbrace{[0,1]}_{\text{BOS}}, \underbrace{[1,0.2]}_{\text{pos1}}, \underbrace{[1,04]}_{\text{pos2}}, \underbrace{[\ ]}_{\text{pos3}}, [\ ], [\ ], [\ ], [\ ], [\ ]]$ (x1, x2, x3, x4, ...)

9 x 2

4. Atención. La atención hace una mezcla distita para cada posición. Cada posición solo mezcla los vectores que puede ver y con esto obtiene hi.

Todas las posiciones tienen el mismo # de cabezas de atención

Posición	Mezcla los vectores...	Resultado
1	x_1 (solo BOS)	h_1
2	x_1, x_2	h_2
3	x_1, x_2, x_3	h_3
...	...	...
6	$x_1, \ldots, x_6$	h_6
...	...	...
9	$x_1, \ldots, x_9$	h_9

→ En la posición 2. cada cabeza reparte su atención entre 2 vectores (x_1, x_2)

→ En la posición 3, cada cabeza reparte su atención entre 3 vectores (x_1, x_2, x_3)

⟹ Cada cabeza realiza su propio cálculo de atención con sus matrices

⟹ El tamaño de h_i no depende del # de cabezas. Solo depende de "d"

Por ejemplo: Con 2 cabezas de atención:

→ cabeza 1: mira al vecino de la izquierda. → W_Q, W_K, W_V

→ cabeza 2: mira al vecino de arriba. → W_Q, W_K, W_V

La posición 2 puede ver a x_1 (Bos) y x_2 (ella misma).

Se deben transformar los vectores x_i en 3 vectores:

$$q_i = W_Q \cdot x_i$$
$$k_j = W_K \cdot x_j$$
$$v_j = W_V \cdot x_j$$

$$W_Q = \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix}, \quad W_K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad W_V = \begin{bmatrix} 1 & 0 \\ 0 & 0.5 \end{bmatrix}$$

La posicion 2 es la que pregunta, solo se necesita su query

$$q_2 = \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} 1 \\ 0.2 \end{bmatrix} = \begin{bmatrix} 1 \\ 0.4 \end{bmatrix}$$

Las posiciones 1 y 2 son las que se comparan y se mezclan. Se necesitan sus keys y sus values:

- Para Bos:

$$k_1 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

$$v_1 = \begin{bmatrix} 1 & 0 \\ 0 & 0.5 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0.5 \end{bmatrix}$$

- Para s_1

$$k_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0.2 \end{bmatrix} = \begin{bmatrix} 1 \\ 0.2 \end{bmatrix}$$

$$v_2 = \begin{bmatrix} 1 & 0 \\ 0 & 0.5 \end{bmatrix} \begin{bmatrix} 1 \\ 0.2 \end{bmatrix} = \begin{bmatrix} 1 \\ 0.1 \end{bmatrix}$$

A. Cuánto se parece a cada uno? (producto punto)

→ Con BOS: $q_2 \cdot k_1 = [1,04] \cdot [0,1] = 0.40$

→ Con S1: $q_2 \cdot k_2 = [1,04] \cdot [1,02] = 1.08$ ⇝ Se parece más a S1

B. Ahora, convertir los parecidos en porcentajes (softmax): primero se divide por $\sqrt{d} = \sqrt{2} = 1.414$

→ Con BOS: $0.4 / 1.414 = 0.283$

→ Con S1: $1.08 / 1.414 = 0.764$

Luego aplicar el softmax (exponencial y luego dividir por la suma)

→ Con BOS: $e^{0.283} = 1.327$

→ Con S1: $e^{0.764} = 2.146$

$1.327 + 2.146 \} 3.473$

→ Peso de BOS: $\frac{1.152}{3.473} = 0.382$

→ Peso de S1: $\frac{2.086}{3.473} = 0.618$

C. Calcular h_i (promedio ponderado) dice que:

"La posición 2 hace todo lo que puede ver (BOS, S1) mezclado según cuánto le importa cada uno."

$h_2 = 0.382 \cdot [0,0.5] + 0,618 \cdot [1, 0.1] = [0.618, 0.253]$

"De lo que ví, el 62% es un espín +1 y el resto (BOS) no aporta información del espín."

No dice nada del valor del espín. Solo lleva información de a quien se miró y donde estaba.

Así, se obtiene un solo vector h_i a partir de los anteriores.
Esto es importante porque la capa de salida solo recibe 1 vector (h_i)
Es decir, cada h_i pasa por la capa de salida y da su probabilidad $q_i(s_i | s_{<i})$

Cómo se relaciona esto con Query, Key y value? →

Paso	Qué hicimos	Quién participó	Nombre de su papel
A. Producto punto	$x_2 \cdot x_1$ y $x_2 \cdot x_2$	x_2: la posición que **pregunta**	**Query**
		x_1, x_2: las posiciones **comparadas**	**Keys**
B. Softmax	Convertir a pesos	—	—
C. Promedio	$0.356 \cdot x_1 + 0.644 \cdot x_2$	x_1, x_2: los vectores **mezclados**	**Values**

Cada posición compara su consulta (query) con las claves (Keys) de las posiciones que puede ver

Papel	Símbolo	Pregunta que responde	En un Transformer real
Query (consulta)	q_i	"¿Qué información **busco**?"	$q_i = W_Q\, x_i$
Key (clave)	k_j	"¿Qué información **ofrezco**?"	$k_j = W_K\, x_j$
Value (valor)	v_j	"Si me eligen, ¿qué **entrego**?"	$v_j = W_V\, x_j$

5. **Capa de salida: Logits** Es un puntaje que la red da a cada opción antes de convertirlo en probabilidad. Puede ser un $\# \in \mathbb{R}$ > ó < 0
Mas grande → mas a favor de esa opción

En este caso hay 2 opciones $\begin{cases} s_i = 1 \\ s_i = -1 \end{cases}$ ∴ hay 2 logits $z^{(1)}$ y $z^{(-1)}$

$z_i = W_{out} \cdot h_i + b$

Matriz de θ $(2 \times d)$

filas = opciones de espín $(1, -1)$

columnas = una columna por cada componente de h

Suponiendo $W_{out} = \begin{bmatrix} -1 & 0 \\ 1 & 0 \end{bmatrix}$ y $b = \begin{bmatrix} 0 \\ 0.1 \end{bmatrix}$

$$z = \begin{bmatrix} -1 & 0 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 0.618 \\ 0.253 \end{bmatrix} + \begin{bmatrix} 0 \\ 0.1 \end{bmatrix} = \begin{bmatrix} -0.618 \\ 0.718 \end{bmatrix} = \begin{bmatrix} z^{(-1)} \\ z^{(+1)} \end{bmatrix}$$

6. Softmax

$$e^{-0.618} = 0.539$$
$$e^{0.718} = 2.050$$
$$\Big\} \; 2.589$$

Con esto, las probabilidades son:

$$q(s_2 = +1) = \frac{2.050}{2.589} = 0.792$$

$$q(s_i = +1) = \frac{e^{z(+1)}}{e^{z(-1)} + e^{z(+1)}}$$

$$q(s_2 = -1) = \frac{0.539}{2.589} = 0.208$$

7. Sacar los Ln:

En cada posición, la softmax da 2 probabilidades. Nos quedamos solo con la del espín que realmente está en la configuración. Es decir, se deben hacer los pasos desde 4. - 6. para cada un de las 9 posiciones:

[1, 1, 1, 1, -1, 1, 1, 1, 1]

Tomamos la probabilidad de este valor → Ln( ) +

↳ Tomamos la probabilidad de este valor → Ln(0.792)

⋮ ↳ Tomamos la probabilidad de este valor → Ln(0.402)

$$l = \ln q_\theta(S) = \sum_{i=1}^{9} \ln q_\theta(s_i | s_{<i})$$

Pos.	Entrada	Predice	h_i	$q(s_i{=}+1)$	Real	Prob. del real	ln
1	BOS	s_1	[0, 0.500]	0.525	+1	0.525	−0.644
2	s_1	s_2	[0.618, 0.253]	0.792	+1	0.792	−0.233
3	s_2	s_3	[0.732, 0.246]	0.827	+1	0.827	−0.190
4	s_3	s_4	[0.787, 0.273]	0.842	+1	0.842	−0.172
5	s_4	s_5	[0.826, 0.317]	0.852	**−1**	**0.148**	**−1.912**
6	s_5	s_6	[−0.249, 0.446]	0.402	+1	0.402	−0.911
7	s_6	s_7	[0.754, 0.439]	0.833	+1	0.833	−0.183
8	s_7	s_8	[0.839, 0.543]	0.855	+1	0.855	−0.156
9	s_8	s_9	[0.903, 0.651]	0.871	+1	0.871	−0.139

+

−4.541

Por tanto, la probabilidad de la configuración:

$$q_\theta(S) = e^{-4.541} = 0.0107$$

Es decir, el Transformer genera esta configuración un 1.07% de las veces.

8. Energía de la configuración

$$E(S) = -J \sum_{\text{vecinos}} s_i s_j - h_{campo} \sum_i s_i$$

- J: Fuerza de interacción entre vecinos
- $J>0$ Ferromagnetismo
- $J<0$ Antiferromagnetismo
- $J=1$ escala natural.

$$S = \begin{pmatrix} 1 & 1 & 1 \\ 1 & -1 & 1 \\ 1 & 1 & 1 \end{pmatrix} \longrightarrow$$ Con $J=1$, $h_{campo}=0$ se tiene

$$E(S) = -\sum_{\langle i,j \rangle} s_i s_j = -\sum_{\langle i,j \rangle} s_{ij}\,(s_{ij+1} + s_{i+1,j})$$

$i=1 \rightarrow$ $= -\sum_j s_{1j}\,(s_{1,j+1} + s_{2,j}) = s_{11}(s_{12}+s_{21}) = (1)(1+1) = 2$

$s_{12}(s_{13}+s_{22}) = (1)(1-1) = 0$

$s_{13}(\cancel{s_{14}}^{s_{11}} + s_{23}) = (1)(1+1) = 2$

$i=2 \rightarrow$ $= -\sum_j s_{2j}\,(s_{2,j+1} + s_{3,j}) = s_{21}(s_{22}+s_{31}) = (1)(-1+1) = 0$

$s_{22}(s_{23}+s_{32}) = (-1)(1+1) = -2$

$s_{23}(\cancel{s_{24}}^{s_{21}} + s_{33}) = (1)(1+1) = 2$

$\vdots$

$$E_1(S) = -\{(2+0+2)+(0-2+2)+(2+2+2)\} = -10$$

9. Calcular la energía Libre $=$ → Se calcula $f(S) = E(S) + T\ln(q_0(S)) = E + T\ell.$; usando $T=2$

$$f(S) = -10 + 2(-4.541) = \boxed{-19.08}$$

Esto en realidad se debe calcular para cada configuración, en este ejemplo solo hay una.

→ Se calcula su promedio: $F[q] = \sum_S q(S)\,[E(S) + T\ln(q_0(S))]$

$$F[q] = \boxed{-16.92}$$

F es la suma de 512 términos. # posible de configuraciones para una red $3\times3 \rightarrow 2^9 = 512$

10. Calcular Promedios del Lote: $\hat{F}_\theta = \frac{1}{M}\sum_{m=1}^{M} f_m$ → Promedio de la energía Libre: indica qué tan bien va el modelo (se grafica) = -17.97 es un estimado del valor exacto (-16.92)

$\hat{U}_\theta = \frac{1}{M}\sum_{m=1}^{M} E_m$ → Promedio de la energía media

$\hat{S}_\theta = \frac{1}{M}\sum_{m=1}^{M} l_m$ → Promedio de la entropía estimada

Comprobar que $\hat{U} - T\hat{S} = \hat{F}$

11. Construir $\mathcal{L}_{aux}$: es un parámetro de ayuda, el cual su derivada es la energía libre

$\mathcal{L}_{aux} = \frac{1}{M}\sum_{m} \underbrace{\text{stop grad}(f_m)}\, l_m$ → Para cada configuración, toma qué tan buena fué comparada con el promedio $(f_m - \hat{F})$ y luego lo multiplica por l_m (log prob. de la configuración)

Toma f_m como constante para que el backward() no lo derive.

Config.	¿Qué tan buena? $(f_m - \bar{f})$	ℓ_m	Producto
A	−1.11 (un poco mejor)	−4.541	5.04
B	−3.93 (mejor)	−1.948	7.66
C	−4.65 (la mejor)	−2.309	10.74
D	+9.71 (la peor)	−7.130	−69.23
		Promedio	$L_{aux} = -11.45$

12. Hacer el Backward(): Pytorch deriva la función $\mathcal{L}_{aux}$ y calcula cómo mover cada parámetro para que las configuraciones buenas suban su probabilidad y la mala la baje

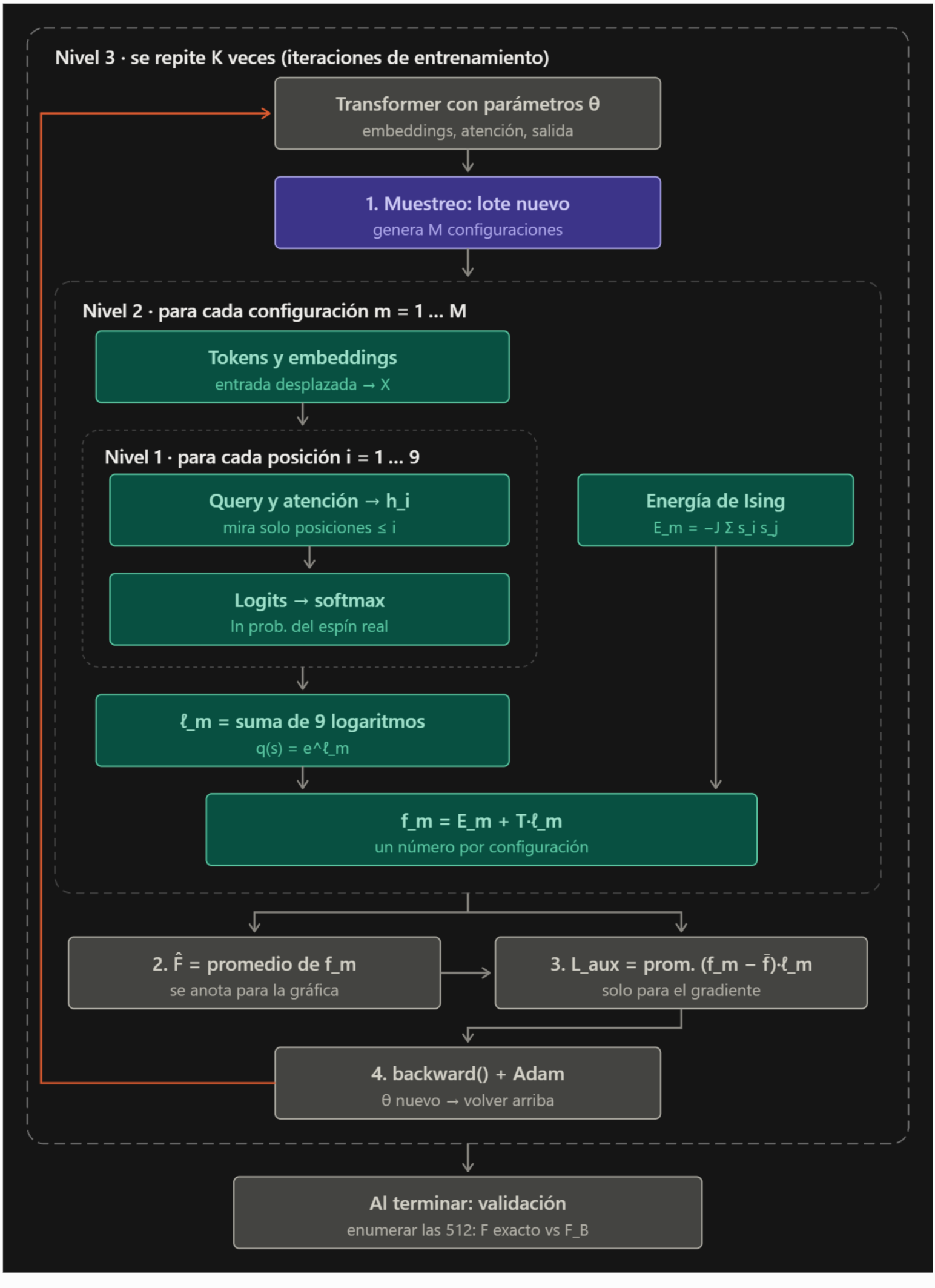
13. Paso de Adam: Mueve los parámetros en la dirección promedio de los gradientes recientes con un tamaño de paso ajustado a la escala de ese parámetro

¿Cómo aprende el Transformer?

A diferencia de un modelo de lenguaje donde el modelo aprende de ejemplos, el Transformer nunca ve configuraciones que sean descritas por la distribución de Bolzmann. Es decir, nunca ve una respuesta correcta.

El Transformer aprende de la siguiente manera:

1. Genera sus propias configuraciones (muestreo)
2. Mide la energía de cada configuración que generó y dice qué tan probable lo hizo el mismo.
3. Calcula $fm = Em + T Im$ y Laura le indica cuales configuraciones debe hacer más probables ó menos probables.
4. El descenso por gradiente ó Adams, ajustan los parámetros en esa dirección


Nivel 3 · se repite K veces (iteraciones de entrenamiento)
Transformer con parámetros θ
embeddings, atención, salida
1. Muestreo: lote nuevo
genera M configuraciones
Nivel 2 · para cada configuración m = 1 … M
Tokens y embeddings
entrada desplazada → X
Nivel 1 · para cada posición i = 1 … 9
Query y atención → h_i
mira solo posiciones ≤ i
Logits → softmax
ln prob. del espín real
Energía de Ising
E_m = −J Σ s_i s_j
ℓ_m = suma de 9 logaritmos
q(s) = e^ℓ_m
f_m = E_m + T·ℓ_m
un número por configuración
2. F̂ = promedio de f_m
se anota para la gráfica
3. L_aux = prom. (f_m − f̄)·ℓ_m
solo para el gradiente
4. backward() + Adam
θ nuevo → volver arriba
Al terminar: validación
enumerar las 512: F exacto vs F_B

1. Configuración 1: [1 , 1 , 1 , 1 , -1 , 1 , 1 , 1 , 1]

2. Asociar un token: [1 , 1 , 1 , 1 , 0 , 1 , 1 , 1 , 1]

3. Entrada desplazada: [2 , 1 , 1 , 1 , 1 , 0 , 1 , 1 , 1]
   BOS S1 S2 S8

4. Embedding de token: cada token se cambia por un vector de d números, buscándolo en una tabla que tiene un vector por tipo de token.
   Hay 3 tipos de vectores porque hay 3 tokens

[2 , 1 , 1 , 1 , 1 , 0 , 1 , 1 , 1]

emb_token = nn.Embedding(vocab, d_model=2) → [[0,1] , [1,0] , [1,0] , [1,0] , [1,0] , [-1,0] , [1,0] , [1,0] , [1,0]]

5. Embedding de Posición: cada posición recibe un vector de d=2 números según su posición buscándolo en una tabla con un vector por casilla

[2 , 1 , 1 , 1 , 1 , 0 , 1 , 1 , 1]

emb_pos = nn.Embedding(N , d_model=2) → [[0,0] , [0,02] , [,] , [,] , [,] , [,] , [,] , [,] , [,]]

6. Suma de los 2 embeddings →

[2 , 1 , 1 , 1 , 1 , 0 , 1 , 1 , 1]

[[0,1] , [1,0] , [1,0] , [1,0] , [1,0] , [-1,0] , [1,0] , [1,0] , [1,0]]

\+ [[0,0] , [0,02] , [,] , [,] , [,] , [,] , [,] , [,] , [,]]

[[0,1] , [1,02] , [,] , [,] , [,] , [,] , [,] , [,] , [,]]

x_1 x_2 x_3 ... x_9

7. Capa de atención: Se transforma cada x_i →

$q_i = W_Q \cdot x_i$

$k_j = W_K \cdot x_j$

$v_j = W_V \cdot x_j$

W_Q, W_K, W_V: Matrices con pesos θ. inicialmente aleatorios

A. Para cada vector x_i se calcula:

por ejemplo para la posición 2 → $q_2 = \begin{bmatrix}1 & 0\\ 0 & 2\end{bmatrix}\begin{bmatrix}1\\ 0.2\end{bmatrix} = \begin{bmatrix}1\\ 0.4\end{bmatrix}$

La posición 2 puede ver el vector Bos (x_1) y ella misma (x_2).

- Para Bos (x_1)

$k_1 = \begin{bmatrix}1 & 0\\ 0 & 1\end{bmatrix}\begin{bmatrix}0\\ 1\end{bmatrix} = \begin{bmatrix}0\\ 1\end{bmatrix}$

$v_1 = \begin{bmatrix}1 & 0\\ 0 & 0.5\end{bmatrix}\begin{bmatrix}0\\ 1\end{bmatrix} = \begin{bmatrix}0\\ 0.5\end{bmatrix}$

- Para Si (x_2)

$k_2 = \begin{bmatrix}1 & 0\\ 0 & 1\end{bmatrix}\begin{bmatrix}1\\ 0.2\end{bmatrix} = \begin{bmatrix}1\\ 0.2\end{bmatrix}$

$v_2 = \begin{bmatrix}1 & 0\\ 0 & 0.5\end{bmatrix}\begin{bmatrix}1\\ 0.2\end{bmatrix} = \begin{bmatrix}1\\ 0.1\end{bmatrix}$

Es decir: para posición 2 → q_2, k_1, v_1, k_2, v_2

para posición 3 → $q_3, k_1, v_1, k_2, v_2, k_3, v_3$

⋮ ⋮

B. Cada q_i se compara con todas las Keys (producto punto y dividir por $\sqrt{d}$)

Se les asignan puntajes y aplicar sobre ellos softmax

Posición 2: $(q_2 \cdot k_1)/\sqrt{d} = 0.283 \xrightarrow{\text{Smax}} \frac{e^{0.283}}{e^{0.283}+e^{0.76}} = 0.382$

$(q_2 \cdot k_2)/\sqrt{d} = 0.764 \xrightarrow{\text{Smax}} \frac{e^{0.764}}{e^{0.283}+e^{0.76}} = 0.618$

Posición 3: $(q_3 \cdot k_1)/\sqrt{d} =$ = #1

$(q_3 \cdot k_2)/\sqrt{d} =$ = #2

$(q_3 \cdot k_3)/\sqrt{d} =$ = #3

Posición 9: $(q_9 \cdot k_1)/\sqrt{d} =$ = #

$(q_9 \cdot k_2)/\sqrt{d} =$ = #

$(q_9 \cdot k_3)/\sqrt{d} =$ = #

$(q_9 \cdot k_9)/\sqrt{d} =$ ⋮

C. Calcular h_i (promedio ponderado) dice que:

Por ejemplo para la posición 2:

"La posición 2 hace todo lo que puede ver (BOS, S1) mezclado según cuánto le importa cada uno."

Pos x_2 : $h_2 = 0.382 \cdot [0, 0.5] + 0.618 \cdot [1, 0.1] = [0.618, 0.253]$

"De lo que ví, el 62% es un espín +1 y el resto (BOS) no aporta información del espín."

No dice nada del valor del espín. Solo lleva información de a quien se miró y donde estaba.

Pos x_3 : $h_3 = \#1 \cdot \text{Vector } v_1 + \#2 \cdot \text{Vector } v_2 + \#3 \cdot \text{Vector } v_3 \longrightarrow h_2 = [h_{21}, h_{22}]$

Pos x_4 : $h_4 = \#1 \cdot \text{Vector } v_1 + \#2 \cdot \text{Vector } v_2 + \#3 \cdot \text{Vector } v_3 + \#4 \cdot \text{Vector } v_4 \longrightarrow h_3 = [h_{31}, h_{32}]$

$\vdots$

$h_9 = [h_{91}, h_{92}]$

8. Capa de salida : Logits — Es un puntaje que la red da a cada opción antes de convertirlo en probabilidad. Puede ser un $\# \in \mathbb{R} > 0$ ó < 0

Mas grande → mas a favor de esa opción

En este caso hay 2 opciones $\begin{cases} S_i = 1 \\ S_i = -1 \end{cases}$ ∴ hay 2 logits $z^{(1)}$ y $z^{(-1)}$

$z_i = W_{out} \cdot h_i + b$ y luego aplicar softmax a cada componente

Matriz de θ $(2 \times d)$

filas = opciones de espín (1, -1)

columnas = una columna por cada componente de h.

Suponiendo $W_{out} = \begin{bmatrix} -1 & 0 \\ 1 & 0 \end{bmatrix}$ y $b = \begin{bmatrix} 0 \\ 0.1 \end{bmatrix}$

$h_2 = [h_{21}, h_{22}]$ $\quad z_2 = \begin{bmatrix} -1 & 0 \\ 1 & 0 \end{bmatrix}\begin{bmatrix} 0.618 \\ 0.253 \end{bmatrix} + \begin{bmatrix} 0 \\ 0.1 \end{bmatrix} = \begin{bmatrix} -0.618 \\ 0.718 \end{bmatrix} = \begin{bmatrix} z^{(-1)} \\ z^{(+1)} \end{bmatrix}$

$h_3 = [h_{31}, h_{32}]$ $\quad z_3 = \begin{bmatrix} z^{(-1)} \\ z^{(+1)} \end{bmatrix}$

$\vdots$

$h_9 = [h_{91}, h_{92}]$ $\quad z_9 = \begin{bmatrix} z^{(-1)} \\ z^{(+1)} \end{bmatrix}$

La dimensión de la capa de salida es entonces $(M, 9, 2)$

9. Aplicar el Softmax sobre cada uno de los logits y sacar el logaritmo.

$$z_2 = \begin{bmatrix} z^{(-1)} \\ z^{(+1)} \end{bmatrix} \xrightarrow{S.max} \begin{bmatrix} q(s_2=-1) = \frac{e^{z(-1)}}{e^{z(-1)} + e^{z(+1)}} \\ q(s_2=+1) = \frac{e^{z(+1)}}{e^{z(-1)} + e^{z(+1)}} \end{bmatrix} \xrightarrow{\ln} \begin{bmatrix} \ln q(s_2=-1) \\ \ln q(s_2=+1) \end{bmatrix}$$

$$z_3 = \begin{bmatrix} z^{(-1)} \\ z^{(+1)} \end{bmatrix} \xrightarrow{S.max} \begin{bmatrix} q(s_3=-1) = \frac{e^{z(-1)}}{e^{z(-1)} + e^{z(+1)}} \\ q(s_3=+1) = \frac{e^{z(+1)}}{e^{z(-1)} + e^{z(+1)}} \end{bmatrix} \xrightarrow{\ln} \begin{bmatrix} \ln q(s_3=-1) \\ \ln q(s_3=+1) \end{bmatrix}$$

$\vdots$

$$z_9 = \begin{bmatrix} z^{(-1)} \\ z^{(+1)} \end{bmatrix} \xrightarrow{S.max} \begin{bmatrix} q(s_9=-1) = \frac{e^{z(-1)}}{e^{z(-1)} + e^{z(+1)}} \\ q(s_9=+1) = \frac{e^{z(+1)}}{e^{z(-1)} + e^{z(+1)}} \end{bmatrix} \xrightarrow{\ln} \begin{bmatrix} \ln q(s_9=-1) \\ \ln q(s_9=+1) \end{bmatrix}$$

10. En cada posición, la softmax da 2 probabilidades. Nos quedamos solo con la del espín que realmente está en la configuración.

$[1, 1, 1, 1, -1, 1, 1, 1, 1]$

Tomamos la probabilidad de este valor → ( ) +

↳ Tomamos la probabilidad de este valor → ( )

$\vdots$ ↳ Tomamos la probabilidad de este valor → ( )

↳ Tomamos la probabilidad de este valor → ( )

$$l = \ln q_\theta(S) = \sum_{i=1}^{9} \ln q_\theta(s_i | s_{<i})$$

11. Calcular la energía de Ising

$$E(s) = -J \sum_{\text{vecinos}} s_i s_j - \cancel{h_{\text{campo}} \sum_i s_i}^{0} - \sum_{\langle ij \rangle} s_i s_j = -\sum_{\langle ij \rangle} s_{ij}(s_{ij_{\text{up}}} + s_{ij_{\text{right}}})$$

⟶ Se le ingresa M configuraciones de N=9 espines

$$\begin{bmatrix} [1\ 1\ 1\ 1\ -1\ 1\ 1\ 1\ 1] \\ [1\ 1\ 1\ 1\ 1\ 1\ 1\ 1\ 1] \\ [-1\ -1\ -1\ -1\ 1\ -1\ -1\ -1\ -1] \\ \vdots \\ [-1\ 1\ -1\ 1\ -1\ 1\ -1\ 1\ -1] \end{bmatrix}_{M \times N}$$

Para 2 configuraciones:

$$[1\ 1\ 1\ 1\ -1\ 1\ 1\ 1\ 1] \xrightarrow{\text{reshape}} \begin{bmatrix} 1 & 1 & 1 \\ 1 & -1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$[1\ 1\ 1\ 1\ 1\ 1\ 1\ 1\ 1] \rightarrow \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Para el Transformer, las configuraciones deben estar en 1D.

M x N

Para la energía, las configuraciones deben estar en 2D

(M, L, L)

Generación de las configuraciones

- Se va a crear un tensor que tiene : torch.full(forma, valor)
  - ⟶ La forma: (M_lote, 1) : M filas y 1 columna
  - ⟶ El valor : se llenan con el valor del token BOS = 2.
  - Tipo de valor, # enteros : dtype = torch.long

Este es el primer elemento de la configuración

Por ejemplo, para un lote con 3 configuraciones y 2 iteraciones:

inicio	iteración 1	iteración 2	...
[[2],	[[2, 1],	[[2, 1, 1],	
[2],	[2, 0],	[2, 0, 0],	
[2]]	[2, 1]]	[2, 1, 1]]	

- Se crea un tensor donde irán guardando los espines generados
  Este tiene tamaño (M x N) : `espines = torch.zeros(M_lote, N, dtype=torch.long)`

```
[[0 0 0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0 0 0]
          ⋮
 [0 0 0 0 0 0 0 0 0 0]]
```

- El loop no recorre configuración por configuración, sino que recorre columna por columna. Las M filas se llenan juntas.

1. Se le pasa al transformer lo que se tiene hasta ahora en cada fila.
   y se le pide que diga la probabilidad del siguiente espin.
   Recordar que el Transformer devuelve los logits.
   `logits = modelo(entrada)`

2. De todos los logits que devuelve el Transformer, se toma el último par:

Configuración 1.

i=0 → Entrada: [BOS] — logits: [[0, 0.1]]

i=1 → Entrada: [BOS, 1] — logits: [[0, 0.1], [0, 1.34]]

i=2 → Entrada: [BOS, 1, 1] — logits: [[0, 0.1], [0, 1.34], [0., 1.8]]

Configuración 2.

i=0 → Entrada: [BOS] — logits: [[0, 0.1]]

i=1 → Entrada: [BOS, 1] — logits: [[0, 0.1], [0, 1.34]]

i=2 → Entrada: [BOS, 1, 1] — logits: [[0, 0.1], [0, 1.34], [0.1, 1.5]]

Configuración 3.

i=0 → Entrada: [BOS] — logits: [[0, 0.1]]

i=1 → Entrada: [BOS, 1] — logits: [[0, 0.1], [0, 1.34]]

i=2 → Entrada: [BOS, 1, 1] — logits: [[0, 0.1], [0, 1.34], [0.1, 1.5]]

S. max → (dim=-1) se aplica horizontalmente

Por ejemplo para i=2 : [0. , 1.8]

[0.1 , 1.5]

forma de los logits: (M, posiciones, 2) = (M, posiciones, [,])

logits [: , -1 , :]

Inicialización del Entrenamiento

- Se crea la variable modelo

modelo = Transformer (N, d_model) ⟹ Esto fija los valores iniciales de los parámetros

El número de parámetros está determinado por d:

parámetros = $3d$ + Nd + $3d^2$ + $2d + 2$ = 3522 parámetros

- $3d$ → Embeddings de tokens
- Nd → Embeddings de posiciones
- $3d^2$ → W_Q, W_K, W_V
- $2d + 2$ → Capa de Salida.

Con d = 32 y N = 9